

Toplantı Tutanaklarından Yazılım Gereksinimi Çıkarma İçin LLM Tabanlı Prompt Araştırması

Araştırmanın odağı ve kısa sonuç

Bu rapor, özellikle **toplantı dökümleri, gereksinim röportajları ve diğer serbest metinlerden yazılım gereksinimi çıkarma** problemine odaklanan çalışmaları derler. En güçlü bulgu şu: literatürde tek adımlı, tek prompt ile çalışan yaklaşımlar yerine, **görevi parçalayan, izlenebilirlik isteyen, çıktıyı yapılandıran ve insan doğrulamasını koruyan** akışlar daha güvenilir görünüyor. Özellikle RECOVER, WER 2025 çalışması, ReqInOne ve Inter2US bu problem için doğrudan incelenmesi gereken çekirdek kaynaklar. ¹

Bir başka net sonuç da şu: prompt mühendisliği bu alanda doğrudan kaliteyi etkiliyor, ama etkisi ölçüte göre değişiyor. Örneğin 2026 tarihli “issue titles to requirements” çalışmasında **few-shot** yaklaşımı gereksinimlerin atomikliğini artırırken, **expert identity** bazı durumlarda doğrulanabilirliği artırsa da tek bir gereksinime odaklanmayı bozabiliyor. Benzer şekilde RECOVER, daha geniş kapsamlı promptların özellikle işlevsel olmayan gereksinimlerde halüsinasyon riskini artırdığını, bu yüzden promptu “yalnızca açıkça söylenen sistem gereksinimleri” ile sınırlamanın faydalı olduğunu gösteriyor. ²

Senin kullanım senaryona en çok benzeyen çizgi şu şekilde özetlenebilir: **transkripti temizle veya parçala, gereksinim adaylarını çıkar, her aday için transcript kanıtı iste, sonra gereksinimi sınıflandır ve backlog ya da SRS formatına dönüştür**. Literatürde tam olarak bu zincire yaklaşan çalışmalar mevcut ve en iyi ilham bunlardan geliyor. ³

En ilgili akademik çalışmalar

RECOVER: Toward Requirements Generation from Stakeholders’ Conversations bu araştırmada en merkezi kaynaktır, çünkü doğrudan paydaş konuşmalarından sistem gereksinimi üretmeye odaklanır. Çalışma, konuşma turlarını önce gereksinim bakımından ilgili olup olmadıklarına göre ayırır, sonra bağlamı temizler ve son aşamada LLM ile gereksinim üretir. Yazarlar bu yaklaşımın gerçek paydaş konuşmalarından gereksinim üretmede umut verici olduğunu, sınıflandırma adımında yüzde 63 precision ve yüzde 77 recall elde ettiğini, kullanıcı değerlendirmesinde de oluşturulan gereksinimlerin doğruluk, kapsayıcılık ve eyleme dönüklük açısından anlamlı kabul gördüğünü rapor eder. Ayrıca geniş promptların işlevsel olmayan gereksinimlerde halüsinasyon üretebildiğini, bu yüzden son promptu açıkça “conversation excerpt içinden system requirements türet” biçimine daralttıklarını belirtirler. ⁴

From Elicitation Interviews to Software Requirements: Evaluating LLM Performance in Requirement Generation senin kullanım durumuna neredeyse bire bir yakındır, çünkü gerçek paydaşlarla yapılan gereksinim görüşmelerinin transkriptlerinden işlevsel ve işlevsel olmayan gereksinim çıkarımını inceler. En önemli tarafı, kullandığı prompt desenini açıkça vermesidir: persona, bağlam, görev, girdi biçimi ve çıktı biçimi. Çalışmada bu şablonla few-shot destekli üretim yapılmış, ayrıca işlevsel ve işlevsel olmayan gereksinimler ayrı biçimde istenmiştir. Sonuçlar proje bazında değişse de, +Lugar örneğinde ChatGPT-4, DeepSeek-V3’e göre daha yüksek precision, recall ve F1 üretmiştir. Bu çalışma, **transkript tabanlı gereksinim çıkarım promptu** tasarlamak için en doğrudan kopyalanabilir akademik örneklerden biridir. ⁵

Inter2US: Automated Alignment between Elicitation Interviews and Requirements doğrudan yeni gereksinim üretmekten çok, üretilmiş kullanıcı hikayelerinin transcript içinde gerçekten desteklenip desteklenmediğini ölçer. Bu, senin sistemi kurarken çok değerlidir, çünkü sadece üretmek yetmez, **metinden gerçekten çıkarılabilir mi** sorusunu da sormak gerekir. Çalışma transkripti üç konuşma turundan oluşan örtüşmeli parçalara ayırıyor ve her kullanıcı hikayesi ile transcript parçası için LLM-as-a-judge kullanarak “bu hikâye bu parçadan gerekçelendirilebilir mi?” sorusunu soruyor. Bu yaklaşım, senin nihai pipeline’ında “faithfulness denetleyicisi” olarak doğrudan kullanılabilir. 6

ReqInOne: A Large Language Model-Based Agent for Software Requirements Specification Generation daha çok genel doğal dilden yapılandırılmış SRS üretmeye odaklanır, fakat meeting transcript ve conversational records gibi girdileri açıkça hedefler. En önemli katkısı, görevi üçe bölmelidir: özetleme, gereksinim çıkarımı ve gereksinim sınıflandırması. Gereksinim çıkarım bileşeninde “trace to source” yani her gereksinim için kaynak ve gerekçe isteyerek halüsinasyonu azaltmayı hedefler. Gereksinim sınıflandırma bileşeninde ise FR ve NFR tanımlarını, NFR alt türlerini ve few-shot örnekleri prompta dahil eder. Uzman değerlendirmesinde ReqInOne’in daha doğru ve daha iyi yapılandırılmış SRS ürettiği, bazı karşılaştırmalarda giriş seviyesindeki gereksinim mühendislerini ve tek adımlı baseline’ı geçtiği raporlanır. Ayrıca ilgili GitHub deposunu da yayımlamışlardır. 7

Evaluating LLM-Based Goal Extraction in Requirements Engineering: Prompting Strategies and Their Limitations doğrudan meeting transcript üzerine kurulu değildir, ama hedef odaklı gereksinim çıkarımında prompt stratejilerinin nereye kadar işe yaradığını çok net gösterir. Çalışma, aktör çıkarımı, yüksek seviye hedef çıkarımı ve düşük seviye hedef çıkarımı için prompt zinciri kurar; ayrıca üretici ve eleştirmen LLM içeren bir geri besleme döngüsü kullanır. Sonuç olarak, son aşamada yüzde 61 doğruluk bildirilmiş ve yöntemin tam otomasyon yerine insanı hızlandıran bir araç olarak daha uygun olduğu söylenmiştir. Zero-shot artı feedback loop kombinasyonunun bazı few-shot kurulumlarını geçmesi de dikkat çekicidir. Bu bulgu, senin senaryonda “çok örnek eklemek her zaman daha iyi değildir” mesajını verir. 8

From issue titles to requirements: an empirical study of large language models and prompt engineering strategies meeting transcript yerine issue başlıklarını kullanır, ama prompt mühendisliğinin etkisini temiz deney tasarımıyla ölçtüğü için çok değerlidir. 150 feature request başlığından 900 gereksinim üretmişler, naive, few-shot ve expert identity promptlarını kıyaslamışlardır. Few-shot her iki modelde de **singularity** ölçütünü düzenli olarak iyileştirirken, expert identity bazı durumlarda doğrulanabilirliği artırmış ama sıklıkla gereksinimin atomikliğini bozmuştur. Bu, transcript tabanlı senaryon için de doğrudan çıkarım verir: gereksinimlerin kısa, tek amaçlı ve test edilebilir olmasını istiyorsan persona katmanını tek başına yetmez, örnekli ve biçimsel rehberlik daha etkilidir. 9

Daha geniş bağlam için iki tarama çalışması da önemlidir. Frontiers’daki sistematik inceleme, LLM’lerin gereksinim çıkarma, analiz ve belirtim üretiminde yükselişte olduğunu; ancak sanayiye yakın gerçek dünya doğrulamalarının hâlâ sınırlı olduğunu vurgular. 2025 tarihli geniş SLR ise 74 birincil çalışmayı inceleyerek LLM4RE araştırmalarının çoğunun elicitation ve validation üzerinde yoğunlaştığını, zero-shot ve few-shot prompting’in baskın olduğunu, fakat karmaşık iş akışlarına gömülü araçların hâlâ az olduğunu raporlar. Bu iki kaynak, neden senin gibi “uçtan uca transcript to requirements” sistemlerinin hâlâ değerli ve açık araştırma alanı olduğunu açıklar. 10

İncelenmeye değer projeler ve GitHub depoları

TaohongZ/ReqInOne en önemli depo adaylarından biridir. Makalenin açıkça işaret ettiği resmi repo budur ve çalışma, prompt şablonları, veri setleri, üretilmiş SRS’ler ve deneysel sonuçların GitHub’da

yayımlandığını belirtir. Eğer sen prompt yazımını sadece teorik değil, deney paketleri üzerinden de incelemek istiyorsan ilk bakılacak repo budur. ¹¹

Apex-CS/AI-dev-reqts-gathering daha pratik, uygulama odaklı bir repodur. Depo README'si meeting transcript, ALM work-item geçmişi ve kod deposu değişikliklerinden "explicit and implicit software requirements" çıkarabildiğini, belirsizlikleri işaretlediğini ve user story ile acceptance criteria üretebildiğini söylüyor. En değerli kısmı, örnek prompt konseptini açıkça vermesi ve JSON-ready çıktı tasarlaması. Eğer hedefin "araştırma makalesi gibi değil, çalışan iç araç" üretmekse bu repo iyi ilham verir. ¹²

Luisarueda1/user-story-generation-skill transcript ve gereksinimlerden sprint-ready user story üretmeye odaklanır. Gereksinim çıkarımı ile backlog oluşturma arasında çok pratik bir köprü kurar. Skill dosyasında meeting transcript ve stakeholder conversation girdilerini kabul eder, önce pain point analizi yapar, sonra INVEST uyumlu user story ve Given/When/Then acceptance criteria üretir. Eğer senin çıktı formatın doğrudan "gereksinim listesi" değil de product backlog ise bu repo çok kullanışlıdır. ¹³

franzvill/action-sync transcript to Jira tickets akışına odaklanan açık kaynak bir projedir. README'ye göre meeting transcript alır, kod tabanı bağlamını ekler ve acceptance criteria ile birlikte Jira ticket üretir. Bu repo akademik olarak çok güçlü kanıt sunmasa da, transcript tabanlı gereksinim ve iş kalemi üretiminin uygulama yüzünü görmek için değerlidir. Özellikle "teknik bağlam ekleme" ve "project memory" katmanları senin sistemine ilham verebilir. ¹⁴

Aniruddh-Mallya/RITA makale ile birlikte yayımlanmış araç reposudur. RITA, çevrim içi kullanıcı geri bildirimlerinden request classification, NFR identification ve requirements specification generation yapan uçtan uca bir araç olarak sunulur ve Jira entegrasyonu da vardır. Bu kaynak meeting transcript yerine app review ve online feedback kullanır, ama "ham metinden gereksinim artefaktı üretme" konusunda doğrudan benzerdir. Özellikle hafif, yerelde çalışabilen açık kaynak modeller üzerine kurulması önemlidir. ¹⁵

madhava20217/LLMs-for-SRS-Prompts doğrudan "Using LLMs in Software Requirements Specifications" makalesinin prompt, context, settings ve dokümanlarını paylaşan reproduksiyon deposudur. Toplantı transkripti değil, daha genel SRS üretimi içindir; yine de promptların nasıl organize edildiğini incelemek için çok yararlı bir deney deposudur. ¹⁶

microsoft/PromptKit doğrudan meeting transcript projesi değildir, ama dikkat çekici biçimde "requirements-elicitation", "author-requirements-doc" ve "requirements-reconciliation" gibi yeniden kullanılabilir prompt bileşenleri içerir. README ve katalogda bunların doğal dilden gereksinim çıkarma, gereksinim dokümanı üretme ve çoklu kaynakları uzlaştırma için tasarlandığı açıkça yazılıdır. Kendi prompt mimarini kurarken bunu "prompt parça kütüphanesi" gibi düşünebilirsin. ¹⁷

Yüksek kaliteli blog, cookbook ve uygulama kaynakları

Mistral AI Cookbook içindeki "Call Transcript-to-PRD-to-Ticket Agent" bu rapordaki en doğrudan teknik uygulama yazısıdır. Bu kaynak, görüşme dökümünü önce PRD'ye, sonra özellik ve teknik gereksinim listesine, ardından Linear ticket'larına dönüştüren bir akış kuruyor. PRD üretimi için transkriptte sıkı hizalanma şartı koyuyor, eksik veya transcript dışı bilgileri yasaklıyor ve iteratif geri bildirim ile belgeyi iyileştiriyor. Eğer sen kısa sürede çalışan bir prototip istiyorsan, bu kaynak mimari olarak çok isabetli bir başlangıç noktasıdır. ¹⁸

OpenAI'nin Meeting Minutes öğreticisi doğrudan yazılım gereksinimi çıkarımı yapmıyor; özet, key point ve action item çıkarımı üzerine kurulu. Buna rağmen transcript ön işleme katmanı için faydalı bir referans olabilir. Yani bu kaynak ana çözüm değil, ama "ham konuşma metnini temiz ve yapılandırılmış bir ara temsile dönüştürme" adımıyla yararlanılabilir. Özellikle sen daha sonra gereksinim çıkarımını ikinci prompt ile yapmak istersen, böyle iki aşamalı kurgu işe yarar. ¹⁹

GitHub Resources'daki spec-driven development yazısı transcript odaklı değildir; ancak gereksinim odaklı, spec-first yazılım geliştirme akışını anlatır. Metnin vurgusu, önce kullanıcı yolculukları ve başarı ölçütlerini içeren ayrıntılı bir spesifikasyon oluşturmak, sonra teknik planı üretmektir. Bu, transcript tabanlı çıkarımı doğrudan anlatmasa da, senin çıkaracağın gereksinimlerin nasıl sonraki tasarım ve implementasyon aşamalarına bağlanacağı konusunda yararlı bir çerçeve sunar. ²⁰

Literatürden çıkan prompt tasarım ilkeleri

Literatürün en güçlü ortak önerisi, **tek seferde "transkriptten tüm gereksinimleri üret"** demekten kaçınmak gerektiğidir. ReqInOne görevi summary, extraction ve classification olarak bölüyor; RECOVER ise classification, processing ve generation adımlarını ayırıyor; Inter2US da son ürünü transcript kanıtlarıyla eşleştiriyor. Yani en iyi desen, üretimi ve doğrulamayı tek prompt içinde eritmek değil, ayrı adımlara bölmektir. ²¹

İkinci ilke, **kaynak dayanağı zorlamaktır**. ReqInOne her çıkarılmış gereksinim için "trace to source" istemenin halüsinasyonu azaltmaya yardım ettiğini söylüyor. RECOVER ise daha geniş promptların transcriptte geçmeyen kalite nitelikleri uydurabildiğini gördüğü için promptu açıkça sistem gereksinimleri ile sınırlandırıyor. Inter2US'un chunk düzeyinde "bu hikaye gerçekten bu transcript parçasından çıkıyor mu" yargıcı da aynı prensibin başka bir uygulamasıdır. Senin promptunda her gereksinim için "kanıt cümlesi", "konuşmacı", "timestamp veya transcript segmenti" istemek bu yüzden çok güçlü bir tasarım kararı olur. ²²

Üçüncü ilke, **çıktıyı biçimsel ve küçük ölçekli tutmaktır**. WER 2025 çalışmasında çıktı formatı açıkça tanımlanıyor ve işlevsel, işlevsel olmayan gereksinimler için yapı öneriliyor. Issue-to-requirements çalışması ise few-shot örneklerin özellikle tekil, atomik gereksinim üretiminde daha iyi olduğunu gösteriyor. User-story-generation-skill de INVEST ve Given/When/Then ile küçük, test edilebilir dil dayatıyor. Bu yüzden serbest paragraf yerine JSON, tablo veya sabit alanlı şema kullanman gerekir. ²³

Dördüncü ilke, **gereksinim türlerini karıştırmamaktır**. RECOVER'ın önemli bulgularından biri, non-functional requirement istediğinde modelin konuşmada geçmeyen kalite nitelikleri uydurma eğilimidir. ReqInOne ise FR, NFR ve NFR alt türlerini ayrı aşamada sınıflandırarak bu problemi yönetir. Senin promptunda önce "gereksinim var mı", sonra "FR mi NFR mi", sonra "hangi alt tür" sorularını ayrı aşamalarda sormak, tek aşamada hepsini istemekten daha güvenilir olur. ²⁴

Beşinci ilke, **prompta örnek verirken ölçüt bilinciyle hareket etmektir**. 2026 tarihli karşılaştırmalı çalışma, few-shot'un singularty yani atomiklik için düzenli fayda verdiğini; expert persona'nın ise doğrulanabilirliği artırırken gereksinimi uzatıp dağıtabildiğini gösteriyor. EASE 2026 goal extraction çalışması da zero-shot artı feedback-loop kurulumunun bazı few-shot kombinasyonlarını geçtiğini söylüyor. Bu yüzden "daha çok örnek = daha iyi" varsayımı burada doğru değildir. Transcript tabanlı görevlerde birkaç çok iyi örnek, bir sürü orta kalite örnekten daha değerlidir. ²⁵

Altıncı ilke, **insanı döngüde tutmaktır**. Hem RECOVER hem goal extraction çalışması hem de sistematik incelemeler, bu alandaki LLM çözümlerinin hâlâ uzman doğrulamasıyla birlikte kullanılmasının daha

doğru olduğunu vurguluyor. En verimli hedef tam otomasyon değil, analisti hızlandıran ve izlenebilir taslak üreten yarı otomasyondur. ²⁶

Başlangıç için önerilen prompt şablonu

Aşağıdaki şablon, bu raporda öne çıkan çalışmalardaki ortak desenlerin sentezidir. Özellikle WER 2025'in persona ve çıktı formatı yaklaşımı, RECOVER'ın transcript dışı çıkarımı sınırlaması, ReqInOne'ın trace-to-source fikri ve Inter2US'un kanıt mantığı birleştirilmiştir. ²⁷

SİSTEM PROMPTU

Sen kıdemli bir yazılım gereksinimleri analistisin.
Görevin, aşağıdaki toplantı veya gereksinim görüşmesi transkriptinden yalnızca metinde açıkça desteklenen yazılım gereksinimlerini çıkarmaktır.

Kurallar:

1. Transcript dışında bilgi uydurma.
2. Her gereksinim için transcript içinden kanıt ver.
3. Bir gereksinim tek bir ihtiyacı anlatsın. Birden fazla ihtiyacı aynı maddede birleştirme.
4. Gereksinim belirsizse bunu "Belirsizlik" olarak işaretle, ama uydurarak tamamlama.
5. Gereksinimleri işlevsel ve işlevsel olmayan olarak ayır.
6. İşlevsel olmayan gereksinim ancak transcriptte açık destek varsa üret.
7. Çıktıyı sadece geçerli JSON olarak ver.

Kullanılacak tanımlar:

- İşlevsel gereksinim: Sistemin ne yapması gerektiğini söyler.
- İşlevsel olmayan gereksinim: Performans, güvenlik, kullanılabilirlik, erişilebilirlik, güvenilirlik, bakım yapılabilirlik, uyumluluk gibi kalite veya kısıtları söyler.

ÇIKTI ŞEMASI

```
{
  "summary": "en fazla 8 cümlelik kısa özet",
  "requirements": [
    {
      "id": "R1",
      "type": "FR | NFR",
      "nfr_subtype": "performance | security | usability | accessibility | reliability | maintainability | compliance | none",
      "requirement_text": "The system shall ...",
      "source_evidence": [
        {
          "speaker": "adı varsa",
          "quote": "kısa destekleyici alıntı veya paraphrase",
          "segment_ref": "transcript bölüm numarası veya zaman damgası varsa"
        }
      ]
    }
  ]
}
```

```

    ],
    "confidence": "high | medium | low",
    "ambiguities": ["..."],
    "clarifying_question": "..."
  }
],
"discarded_candidates": [
  {
    "candidate": "...",
    "reason": "yeterli kanıt yok | gereksinim değil | fazla belirsiz"
  }
]
}

```

KULLANICI GİRDİSİ

Proje bağlamı:
{project_context}

Transcript:
{transcript}

Bu temel promptu tek başına kullanabilirsin, ama araştırma bulgularına göre daha iyi sonuç için bunu üç adıma bölmek daha doğru olur: önce kısa özet ve konu segmentasyonu, sonra gereksinim adayları çıkarımı, son olarak transcript kanıtı ile doğrulama ve FR/NFR sınıflandırması. ReqInOne'ın modüler mimarisi, RECOVER'ın çok adımlı iş akışı ve Inter2US'un chunk-level doğrulama mantığı bu parçalı tasarımı güçlü biçimde destekliyor. ²¹

Pratikte ilk prototip için en sağlam kısa yol şu görünür: transcripti konuşmacı dönüşlerine göre parçalara ayır, örtüşmeli küçük segmentler oluştur, her segmentten gereksinim adayları çıkar, sonra her gereksinimi transcript segmentleriyle eşleştir ve ancak kanıt varsa son listeye al. Bu yaklaşım, özellikle uzun toplantılarda bağlam kaybını ve “makul ama yanlış” çıkarımları azaltır. ²⁸

Seçilmiş kaynak haritası

Aşağıdaki kaynaklar en yüksek öncelikli inceleme sırasını temsil eder.

- **RECOVER: Toward Requirements Generation from Stakeholders' Conversations.** Paydaş konuşmalarından otomatik gereksinim üretimi için en doğrudan akademik çekirdek kaynak. Promptu daraltma, adımlı pipeline ve kalite değerlendirmesi içeriyor. ²⁹
- **From Elicitation Interviews to Software Requirements.** Gerçek gereksinim görüşmesi transkriptlerinden FR ve NFR çıkarımı için açık prompt şablonu veriyor. ³⁰
- **ReqInOne paper ve repo.** Meeting transcripts dahil doğal dilden yapılandırılmış SRS üretimi, trace-to-source ve modüler prompt tasarımı sunuyor. ³¹
- **Inter2US.** Transcriptten üretilen gereksinim veya user story'lerin gerçekten transcript tarafından desteklenip desteklenmediğini doğrulamak için ideal. ⁶
- **Mistral Cookbook, Transcript-to-PRD-to-Ticket.** Hızlı prototip için en uygulanabilir resmi teknik yazı. Transcriptten PRD, teknik gereksinim ve ticket üretimine gidiyor. ³²
- **Apex-CS/AI-dev-reqts-gathering.** İç araç benzeri, meeting transcript ve ALM verisinden gereksinim çıkaran pratik repo. ³³

- **Luisarueda1/user-story-generation-skill**. Transcriptten backlog-ready user story ve acceptance criteria üretmek için pratik prompt varlığı. ³⁴
- **RTA**. Online user feedback tabanlı olsa da, ham metinden sınıflandırma, gereksinim belirtimi ve Jira entegrasyonu kuran uçtan uca araç. ³⁵
- **Issue titles to requirements**. Prompt stratejilerinin kalite ölçütlerine göre etkisini ölçen en temiz deneysel kaynaklardan biri. ³⁶
- **LLM4RE systematic literature review**. Alanın genel resmini, hangi prompt stratejilerinin baskın olduğunu ve araç boşluklarını anlamak için iyi çerçeve. ³⁷

Açık sorular ve sınırlamalar

Bu alanda doğrudan **meeting transcript to software requirements** yapan kaynak sayısı artıyor, ama hâlâ sınırlı. Özellikle açık kaynak, iyi belgelenmiş, promptlarını ve veri paketlerini tam paylaşan projeler çok fazla değil. Sistematik inceleme de araçların çoğunun hâlâ “out of the box” ve kontrollü ortam odaklı olduğunu, gerçek dünya entegrasyonlarının az kaldığını vurguluyor. Bu yüzden elindeki en iyi strateji, tek bir kaynağı kopyalamak değil, yukarıdaki kaynaklardan ortak desen çıkarıp kendi veri ve domainine göre uyarlamak olacaktır. ³⁸

Ayrıca transcript tabanlı gereksinim çıkarımında en kritik teknik riskler şunlardır: konuşmada dağınık ifade edilen ihtiyaçların tekil gereksinime dönüştürülmesi, işlevsel olmayan gereksinimlerin “makul tahmin” ile uydurulması, ve transcriptte gerçekten olmayan şeylerin toplantı bağlamından varmış gibi yazılması. RECOVER, ReqInOne ve Inter2US’un değerli olmasının sebebi tam olarak bu riskleri azaltmaya çalışmalarıdır. ³⁹

1 3 4 24 26 29 39 [arxiv.org](https://arxiv.org/pdf/2411.19552)

<https://arxiv.org/pdf/2411.19552>

2 9 25 36 From issue titles to requirements: an empirical study of large language models and prompt engineering strategies | Requirements Engineering | Springer Nature Link

<https://link.springer.com/article/10.1007/s00766-026-00462-z>

5 23 27 30 [werpapers.dimap.ufrn.br](https://werpapers.dimap.ufrn.br/papers/WER2025/wer202511.pdf)

<https://werpapers.dimap.ufrn.br/papers/WER2025/wer202511.pdf>

6 28 Automated Alignment between Elicitation Interviews and Requirements

<https://arxiv.org/html/2510.08622v2>

7 11 21 22 31 ReqInOne: A Large Language Model-Based Agent for Software Requirements Specification Generation

<https://arxiv.org/html/2508.09648v1>

8 Evaluating LLM-Based Goal Extraction in Requirements Engineering: Prompting Strategies and Their Limitations

<https://arxiv.org/pdf/2604.22207>

10 Frontiers | Research directions for using LLM in software requirement engineering: a systematic review

<https://www.frontiersin.org/journals/computer-science/articles/10.3389/fcomp.2025.1519437/full>

12 33 GitHub - Apex-CS/AI-dev-reqts-gathering · GitHub

<https://github.com/Apex-CS/AI-dev-reqts-gathering>

- 13 34 GitHub - Luisarueda1/user-story-generation-skill: Transform requirements & meeting transcripts into INVEST-compliant user stories with acceptance criteria. A portable Claude AI skill. · GitHub
<https://github.com/Luisarueda1/user-story-generation-skill>
- 14 GitHub - franzvill/action-sync: AI tool that turns meeting transcripts into Jira tickets. Claude analyzes your meetings, checks your codebase for context, and creates tickets with technical details. Can also answer questions about your project and work on tickets autonomously. · GitHub
<https://github.com/franzvill/action-sync>
- 15 35 RITA: A Tool for Automated Requirements Classification and Specification from Online User Feedback
<https://arxiv.org/html/2601.11362v1>
- 16 madhava20217/LLMs-for-SRS-Prompts
https://github.com/madhava20217/LLMs-for-SRS-Prompts?utm_source=chatgpt.com
- 17 GitHub - microsoft/PromptKit: Agentic prompts are the most important code you're not engineering. PromptKit fixes that — composable, version-controlled prompt components (personas, protocols, formats, templates) that snap together into reliable, repeatable prompts for bug investigation, design docs, code review, security audits, and more. Works with any LLM. · GitHub
<https://github.com/microsoft/PromptKit>
- 18 32 Call Transcript-to-PRD-to-Ticket Agent: Converting Meeting Transcripts to Linear Tickets using Mistral AI LLMs - Mistral AI Cookbook | Mistral Docs
https://docs.mistral.ai/resources/cookbooks/mistral-agents-non_framework-transcript_linearticket_agent-transcripttolinearticketagent
- 19 Meeting minutes | OpenAI API
https://developers.openai.com/api/docs/tutorials/meeting-minutes?utm_source=chatgpt.com
- 20 Spec-driven development with AI: Get started with a new ...
https://resources.github.com/increasing-collaborative-development-with-ai/?utm_source=chatgpt.com
- 37 38 Large Language Models (LLMs) for Requirements Engineering (RE): A Systematic Literature Review
<https://arxiv.org/html/2509.11446v1>